

8. Metódusok (Függvények) C#-ban

Mi az a metódus?

A **metódus** (más néven függvény) egy **újrafelhasználható kódblokk**, amely egy konkrét feladatot hajt végre. A metódusok segítségével a kódunkat kisebb, kezelhető részekre bonthatjuk.

Miért hasznosak a metódusok?

- **Újrafelhasználhatóság:** Egyszer megírjuk, sokszor használjuk
- **Kódszervezés:** Áttekinthetőbb, strukturáltabb kód
- **Karbantarthatóság:** Könnyebb a hibakeresés és módosítás
- **Olvashatóság:** A kód érthetőbb lesz

```
// Példa: Metódus nélkül (ismétlődő kód)
Console.WriteLine("Hello, Anna!");
Console.WriteLine("Hello, Béla!");
Console.WriteLine("Hello, Cecil!");

// Metódussal (újrafelhasználható kód)
void Udvozles(string nev)
{
    Console.WriteLine($"Hello, {nev}!");
}

Udvozles("Anna");    // Meghívjuk a metódust
Udvozles("Béla");
Udvozles("Cecil");
```

Metódus szerkezete

Egy metódusnak több részből áll:

```
// ÁLTALÁNOS SZERKEZET:
// [hozzáférési szint] [visszatérési típus] [metódus név]([paraméterek])
// {
//     [metódus törzse - utasítások]
//     [return érték;] // ha van visszatérési érték
// }

// KONKRÉT PÉLDA:
static int Osszead(int a, int b)
{
    int eredmeny = a + b;
    return eredmeny;
}
```

```
// Részletes magyarázat:  
// - static: később részletezzük (egyelőre mindig írjuk ki)  
// - int: visszatérési típus (ez a metódus egy int számot ad vissza)  
// - Osszead: a metódus neve (nagy betűvel kezdődik)  
// - (int a, int b): paraméterek (bemenetek) - két darab int típusú változó  
// - { ... }: metódus törzse - ide jönnek az utasítások  
// - return: visszaadja az eredményt (kilép a metódusból)
```

Metódus részei részletesen

```
// 1. HOZZÁFÉRÉSI SZINT (access modifier)  
public    // Bárhonnan elérhető  
private   // Csak az osztályon belülről elérhető  
static    // Az osztályhoz tartozik, nem példányhoz  
  
// 2. VISSZATÉRÉSI TÍPUS (return type)  
void       // Nem ad vissza semmit  
int        // Egész számot ad vissza  
string     // Szöveget ad vissza  
bool       // Logikai értéket ad vissza  
double     // Lebegőpontos számot ad vissza  
  
// 3. METÓDUS NÉV (method name)  
// - Nagy betűvel kezdődik (PascalCase)  
// - Kifejező, érthető név  
// - Ige formában (mit csinál?)  
  
// 4. PARAMÉTEREK (parameters)  
// - Bemenetek a metódusnak  
// - Vesszővel elválasztva  
// - Típus + név formátumban  
  
// 5. METÓDUS TÖRZSE (method body)  
// - Kapcsos zárójelek között  
// - Itt vannak az utasítások
```

Paraméter nélküli metódusok

A legegyszerűbb metódusok: nincs bemenet, nincs kimenet.

```
// void = nem ad vissza semmit  
// üres zárójelek = nincs paramétere  
static void Udvozles()  
{  
    // A metódus törzse  
    Console.WriteLine("Szia! Üdvözöllek!");  
    Console.WriteLine("Hogy vagy?");  
}
```

```

}

// HASZNÁLAT (hívás):
Udvozles(); // Meghívjuk a metódust
// Kimenet:
// Szia! Üdvözöllek!
// Hogy vagy?

// Többször is meghívhatjuk
Udvozles(); // Újra lefut
Udvozles(); // Még egyszer

// Gyakorlati példa
static void FejezetKezdes()
{
    Console.WriteLine("=====");
    Console.WriteLine("    ÚJ FEJEZET    ");
    Console.WriteLine("=====");
    Console.WriteLine();
}

// Használat
FejezetKezdes(); // Kiírja a szép fejléct
Console.WriteLine("Ez az első fejezet tartalma.");

FejezetKezdes(); // Újra kiírja
Console.WriteLine("Ez a második fejezet tartalma.");

```

Több paraméter nélküli metódus példa

```

static void Menu()
{
    // Kiírja a menüt
    Console.WriteLine("=== MENÜ ===");
    Console.WriteLine("1. Új játék");
    Console.WriteLine("2. Betöltés");
    Console.WriteLine("3. Beállítások");
    Console.WriteLine("4. Kilépés");
    Console.WriteLine("=====");
}

static void VonalRajzolas()
{
    // Rajzol egy vonalat
    Console.WriteLine(new string('-', 40));
}

static void ProgramIndit()
{
    // Program induló képernyője
    Console.Clear(); // Törli a képernyőt
}

```

```
    Console.WriteLine("*** PROGRAM INDÍTÁSA ***");
    VonalRajzolas(); // Egy metódus hívhat másik metódust!
    Console.WriteLine("Töltés...");
    System.Threading.Thread.Sleep(1000); // Vár 1 másodpercet
}

// Használat
ProgamIndit();
Menu();
```

Paraméterekkel rendelkező metódusok

A paraméterek a metódusnak átadott **bemeneti értékek**. A metódus ezekkel dolgozik.

Egy paraméter

```
// Egy paraméteres metódus
static void Udvozles(string nev)
{
    // A "nev" paraméter egy változó, amit a metódus használhat
    Console.WriteLine($"Szia, {nev}!");
    Console.WriteLine($"Örülök, hogy itt vagy, {nev}!");
}

// MEGHÍVÁS (calling):
Udvozles("Anna"); // A "Anna" átadódik a nev paraméternek
// Kimenet:
// Szia, Anna!
// Örülök, hogy itt vagy, Anna!

Udvozles("Béla"); // Most "Béla" lesz a nev értéke
// Kimenet:
// Szia, Béla!
// Örülök, hogy itt vagy, Béla!

// FONTOS: A paraméter típusának megfelelő értéket kell átadni!
// Udvozles(123); // HIBA! int-et adunk át string helyett

// Gyakorlati példa: Születésnap üdvözlés
static void Szuletesnapikövetkezo(string nev, int kor)
{
    Console.WriteLine($"Kedves {nev}!");
    Console.WriteLine($"Boldog {kor}. születésnapot kívánunk!");
    Console.WriteLine("Sok boldogságot és egészséget!");
}

// Használat
Szuletesnapikövetkezo("Anna", 25);
// Kimenet:
// Kedves Anna!
```

```
// Boldog 25. születésnapot kívánunk!  
// Sok boldogságot és egészséget!
```

Több paraméter

```
// Két paraméteres metódus  
static void KiirNevet(string vezeteknev, string keresztnév)  
{  
    // A paraméterek SORRENDJE számít!  
    Console.WriteLine($"Teljes név: {vezeteknev} {keresztnév}");  
}  
  
// MEGHÍVÁS:  
KiirNevet("Kovács", "Anna");  
// Kimenet: Teljes név: Kovács Anna  
  
KiirNevet("Nagy", "Béla");  
// Kimenet: Teljes név: Nagy Béla  
  
// FONTOS: A sorrend számít!  
KiirNevet("Anna", "Kovács"); // Ez fordítva lesz!  
// Kimenet: Teljes név: Anna Kovács (valószínűleg nem ezt akarod)  
  
// Három paraméteres metódus  
static void SzemelyAdatok(string nev, int kor, string varos)  
{  
    // Mindhárom paramétert használjuk  
    Console.WriteLine("=== SZEMÉLYI ADATOK ===");  
    Console.WriteLine($"Név: {nev}");  
    Console.WriteLine($"Életkor: {kor} év");  
    Console.WriteLine($"Város: {varos}");  
    Console.WriteLine("=====");  
}  
  
// Használat  
SzemelyAdatok("Kovács Anna", 28, "Budapest");  
// Kimenet:  
// === SZEMÉLYI ADATOK ===  
// Név: Kovács Anna  
// Életkor: 28 év  
// Város: Budapest  
// =====  
  
// Különböző típusú paraméterek  
static void TermekInfo(string nev, decimal ar, int keszlet, bool kapahto)  
{  
    Console.WriteLine($"Termék: {nev}");  
    Console.WriteLine($"Ár: {ar:N0} Ft");  
    Console.WriteLine($"Készlet: {keszlet} db");  
    Console.WriteLine($"Kapható: {(kapahto ? "Igen" : "Nem")}");  
    Console.WriteLine();  
}
```

```
}

// Használat
TermekInfo("Laptop", 250000, 5, true);
TermekInfo("Egér", 3500, 0, false);
```

Paraméterek működése - Részletesen

```
// FONTOS MEGÉRTENI: Mi történik paraméter átadáskor?

static void DuplazMeg(int szam)
{
    // 1. A "szam" paraméter megkapja az átadott érték MÁSOLATÁT
    Console.WriteLine($"Metódus elején: {szam}");

    // 2. A metódus módosítja a saját másolatát
    szam = szam * 2;

    // 3. A módosított érték kiírása
    Console.WriteLine($"Metódus végén: {szam}");
} // 4. A metódus véget ér, a "szam" paraméter megszűnik

// Használat
int eredeti = 5;
Console.WriteLine($"Hívás előtt: {eredeti}"); // 5

DuplazMeg(eredeti);
// Kimenet:
// Metódus elején: 5
// Metódus végén: 10

Console.WriteLine($"Hívás után: {eredeti}"); // Még mindig 5!
// Az "eredeti" változó NEM VÁLTOZOTT!

// MIÉRT?
// - Az "eredeti" ÉRTÉKE (5) átmásolódott a "szam" paraméterbe
// - A metódus a saját másolatát módosította
// - Az eredeti változó érintetlen maradt
// - Ez az "érték szerinti átadás" (pass by value)

// További példa
static void Valtoztat(string szoveg)
{
    Console.WriteLine($"Kapott szöveg: {szoveg}");
    szoveg = "Megváltoztatva!"; // Csak a másolatot módosítjuk
    Console.WriteLine($"Módosított: {szoveg}");
}

string eredeti2 = "Eredeti szöveg";
Console.WriteLine($"Előtte: {eredeti2}");
```

```
Valtoztat(eredeti2);  
Console.WriteLine($"Utána: {eredeti2}"); // Még mindig "Eredeti szöveg"
```

Visszatérési értékkel rendelkező metódusok

A **return** kulcsszóval **értéket adhatunk vissza** a metódusból.

Egyszerű visszatérési érték

```
// A metódus VISSZAAD egy int típusú értéket  
static int Osszead(int a, int b)  
{  
    int eredmeny = a + b; // Kiszámoljuk az összeget  
    return eredmeny;      // VISSZAADJUK az eredményt  
} // A metódus itt véget ér  
  
// HASZNÁLAT - el kell kapnunk a visszatérési értéket!  
int osszeg = Osszead(10, 20); // osszeg = 30  
Console.WriteLine($"Az összeg: {osszeg}");  
  
// Vagy rögtön felhasználjuk  
Console.WriteLine($"5 + 7 = {Osszead(5, 7)}"); // Kiírja: 5 + 7 = 12  
  
// Rövidebb változat (kifejezés test)  
static int Osszead2(int a, int b)  
{  
    return a + b; // Egyből visszaadjuk  
}  
  
// Még rövidebb (expression-bodied method, C# 6.0+)  
static int Osszead3(int a, int b) => a + b;  
  
// Mindhárom ugyanazt csinálja!
```

Visszatérési érték típusai

```
// int visszatérési érték  
static int Szoroz(int a, int b)  
{  
    return a * b;  
}  
  
// double visszatérési érték  
static double Oszto(double a, double b)  
{  
    if (b == 0)  
    {  
        Console.WriteLine("Hiba: nullával nem lehet osztani!");  
    }  
}
```

```

        return 0; // Hibaeset - visszaadunk 0-t
    }
    return a / b;
}

// string visszatérési érték
static string TeljesNev(string vezeteknev, string keresztnév)
{
    string teljes = $"{vezeteknev} {keresztnév}";
    return teljes;
}

// bool visszatérési érték
static bool ParosE(int szam)
{
    return szam % 2 == 0; // true ha páros, false ha páratlan
}

// HASZNÁLAT:
int szorzat = Szoroz(5, 6);           // 30
double hanydos = Oszt(10, 3);         // 3.333...
string nev = TeljesNev("Kovács", "Anna"); // "Kovács Anna"
bool paros = ParosE(8);               // true

// Felhasználás feltételben
if (ParosE(10))
{
    Console.WriteLine("A 10 páros szám!");
}

```

return működése - Részletesen

```

// A return AZONNAL KILÉP a metódusból!
static int ElsoNagyobb(int a, int b)
{
    Console.WriteLine("Metódus kezdete");

    if (a > b)
    {
        Console.WriteLine("Az 'a' nagyobb");
        return a; // KILÉPÜNK! Az alatta lévő kód NEM fut le!
    }

    Console.WriteLine("A 'b' nagyobb vagy egyenlő");
    return b; // KILÉPÜNK innen!

    // Console.WriteLine("Ez sosem fut le!"); // Ez a sor SOHA nem hajtódik
    // végre!
}

// Használat

```

```
int nagyobb = ElsoNagyobb(10, 5);
Console.WriteLine($"Nagyobb: {nagyobb}");

// Több return is lehet egy metódusban
static string Ertekeles(int pontszam)
{
    if (pontszam >= 90)
        return "Jeles"; // Ha teljesül, itt kilép

    if (pontszam >= 80)
        return "Jó"; // Ha teljesül, itt kilép

    if (pontszam >= 70)
        return "Közepes"; // Ha teljesül, itt kilép

    if (pontszam >= 60)
        return "Elégéses"; // Ha teljesül, itt kilép

    return "Elégtelen"; // Csak ide jut el, ha egyik feltétel sem teljesült
}

// void metódusban is használható return (kilépésre)
static void Ellenorzes(int szam)
{
    if (szam < 0)
    {
        Console.WriteLine("Hiba: negatív szám!");
        return; // Kilépünk, nem csinálunk semmit
    }

    // Ha nem negatív, folytatjuk
    Console.WriteLine($"A szám: {szam}");
    Console.WriteLine($"A négyzete: {szam * szam}");
}
```

Gyakorlati példák visszatérési értékkel

```
// Átlag számítás
static double Atlag(int a, int b, int c)
{
    double osszeg = a + b + c;
    double atlag = osszeg / 3;
    return atlag;
}

// Használat
double eredmeny = Atlag(80, 90, 85);
Console.WriteLine($"Átlag: {eredmeny:F2}"); // Átlag: 85.00

// Maximum keresés
static int Maximum(int a, int b, int c)
```

```
{
    int max = a; // Feltételezzük, hogy 'a' a legnagyobb

    if (b > max)
        max = b;

    if (c > max)
        max = c;

    return max;
}

// Használat
int legnagyobb = Maximum(45, 78, 23);
Console.WriteLine($"Legnagyobb: {legnagyobb}"); // 78

// Szám ellenőrzés
static bool ErvenyesSzam(int szam, int min, int max)
{
    return szam >= min && szam <= max;
}

// Használat
if (ErvenyesSzam(50, 1, 100))
{
    Console.WriteLine("Az 50 érvényes szám 1 és 100 között!");
}

// Email validálás (egyszerűsített)
static bool ErvenyesEmail(string email)
{
    // Egyszerű ellenőrzés - tartalmaz @ jelet és pontot
    if (string.IsNullOrEmpty(email))
        return false;

    if (!email.Contains("@"))
        return false;

    if (!email.Contains("."))
        return false;

    return true;
}

// Használat
string email = "teszt@example.com";
if (ErvenyesEmail(email))
{
    Console.WriteLine("Érvényes email cím!");
}
else
{
    Console.WriteLine("Hibás email cím!");
}
```

Alapértelmezett paraméterek (Optional Parameters)

A paramétereknek **alapértelmezett értéket** adhatunk. Ha nem adunk át értéket, az alapértelmezett használdik.

```
// Az "ar" paraméternek van alapértelmezett értéke
static void Vasarlas(string termék, int mennyiség = 1, decimal ar = 0)
{
    Console.WriteLine($"Termék: {termék}");
    Console.WriteLine($"Mennyiség: {mennyiség} db");
    Console.WriteLine($"Ár: {ar:N0} Ft");
    Console.WriteLine($"Összesen: {mennyiség * ar:N0} Ft");
    Console.WriteLine();
}

// KÜLÖNBÖZŐ HÍVÁSI MÓDOK:

// 1. Minden paramétert megadunk
Vasarlas("Kenyér", 2, 450);
// Kimenet:
// Termék: Kenyér
// Mennyiség: 2 db
// Ár: 450 Ft
// Összesen: 900 Ft

// 2. Az utolsó paramétert kihagyjuk - alapértelmezett érték lesz (0)
Vasarlas("Tej", 3);
// Kimenet:
// Termék: Tej
// Mennyiség: 3 db
// Ár: 0 Ft
// Összesen: 0 Ft

// 3. Az utolsó kettőt kihagyjuk - mindkettő alapértelmezett lesz
Vasarlas("Tojás");
// Kimenet:
// Termék: Tojás
// Mennyiség: 1 db
// Ár: 0 Ft
// Összesen: 0 Ft

// FONTOS SZABÁLYOK:
// 1. Az alapértelmezett paramétereknek a VÉGÉN kell lenniük!
// static void Rossz(int a = 5, int b) { } // HIBA! Az alapértelmezett előbb van

// 2. Helyes sorrend:
static void Jo(int a, int b = 5, int c = 10)
{
    Console.WriteLine($"a={a}, b={b}, c={c}");
}
```

```
}

Jo(1);           // a=1, b=5, c=10
Jo(1, 2);        // a=1, b=2, c=10
Jo(1, 2, 3);     // a=1, b=2, c=3

// Gyakorlati példa: Köszönés különböző napszakokban
static void Udvozol(string nev, string napszak = "")
{
    if (string.IsNullOrEmpty(napszak))
    {
        Console.WriteLine($"Szia, {nev}!");
    }
    else
    {
        Console.WriteLine($"Jó {napszak}-ot, {nev}!");
    }
}

Udvozol("Anna");           // Szia, Anna!
Udvozol("Béla", "reggel");  // Jó reggel-ot, Béla!
Udvozol("Cecil", "estét");  // Jó estét-ot, Cecil!

// Téglalap területe (lehet négyzet is)
static double Terulet(double a, double b = 0)
{
    // Ha b nincs megadva (0), akkor négyzet
    if (b == 0)
        b = a;

    return a * b;
}

Console.WriteLine($"Négyzet (5x5): {Terulet(5)} cm²"); // 25
Console.WriteLine($"Téglalap (4x6): {Terulet(4, 6)} cm²"); // 24
```

Nevesített paraméterek (Named Arguments)

A paramétereket **név szerint** is átadhatjuk, nem csak sorrend szerint.

```
// Egy metódus sok paraméterrel
static void SzemelyiAdatok(string nev, int kor, string varos, string email)
{
    Console.WriteLine($"Név: {nev}");
    Console.WriteLine($"Kor: {kor}");
    Console.WriteLine($"Város: {varos}");
    Console.WriteLine($"Email: {email}");
}

// HAGYOMÁNYOS HÍVÁS (sorrend szerint):
```

```
SzemelyiAdatok("Anna", 25, "Budapest", "anna@example.com");

// NEVESÍTETT PARAMÉTEREK (név szerint):
SzemelyiAdatok(
    nev: "Béla",
    kor: 30,
    varos: "Debrecen",
    email: "bela@example.com"
);

// ELŐNY: A sorrend TETSZŐLEGES lehet!
SzemelyiAdatok(
    email: "cecil@example.com", // Email elsőnek
    nev: "Cecil",                // Név másodiknak
    varos: "Szeged",            // Város harmadiknak
    kor: 28                     // Kor negyediknek
);
// Működik! A sorrend nem számít!

// KOMBINÁLHATÓ az alapértelmezett paraméterekkel
static void Rendeles(string termék, int mennyiség = 1, decimal ar = 0, string szin
= "fekete")
{
    Console.WriteLine($"{termék} - {szin} szín, {mennyiség} db, {ar} Ft");
}

// Csak néhány paramétert adunk meg név szerint
Rendeles(
    termék: "Póló",
    szin: "piros",
    ar: 2500
    // mennyiség kihagytuk - alapértelmezett (1) lesz
);

// Gyakorlati példa: HTML generálás
static void HtmlLink(string url, string szoveg, bool ujAblak = false, string
stilus = "")
{
    string target = ujAblak ? " target='_blank'" : "";
    string style = !string.IsNullOrEmpty(stilus) ? $" style='{stilus}'" : "";

    Console.WriteLine($"<a href='{url}'{target}{style}>{szoveg}</a>");
}

// Könnyebb olvasni így:
HtmlLink(
    url: "https://google.com",
    szoveg: "Google",
    ujAblak: true
);

HtmlLink(
    szoveg: "Facebook",
    url: "https://facebook.com",
```

```
        stilus: "color: blue;"
    );
```

ref paraméter - Referencia szerinti átadás

A **ref** kulcsszóval a paraméter **referenciáját** adjuk át, nem a másolatát. A metódus módosíthatja az eredeti változót!

```
// NORMÁL (érték szerinti átadás) - NEM VÁLTOZIK az eredeti
static void DuplazMeg(int szam)
{
    szam = szam * 2; // Csak a másolatot módosítjuk
    Console.WriteLine($"Metódusban: {szam}");
}

int ertek = 5;
Console.WriteLine($"Előtte: {ertek}"); // 5
DuplazMeg(ertek); // Metódusban: 10
Console.WriteLine($"Utána: {ertek}"); // 5 (nem változott!)

// ref PARAMÉTER - MEGVÁLTOZIK az eredeti!
static void DuplazMegRef(ref int szam)
{
    // A "szam" most az eredeti változóra mutat!
    szam = szam * 2; // Az EREDETI változót módosítjuk!
    Console.WriteLine($"Metódusban: {szam}");
}

int ertek2 = 5;
Console.WriteLine($"Előtte: {ertek2}"); // 5
DuplazMegRef(ref ertek2); // FONTOS: "ref" szó kell a hívásnál is!
// Metódusban: 10
Console.WriteLine($"Utána: {ertek2}"); // 10 (megváltozott!)

// FONTOS SZABÁLYOK a ref-nél:
// 1. A változót INICIALIZÁLNI KELL hívás előtt
int x; // Nincs értéke
// DuplazMegRef(ref x); // HIBA! Az x-nek még nincs értéke!

int y = 10; // Van értéke
DuplazMegRef(ref y); // OK!

// 2. ref szó kell mind a metódus definíciónál, mind a hívásnál

// Gyakorlati példa: Csere metódus
static void Csere(ref int a, ref int b)
{
    // Az 'a' és 'b' kicserélése
    int temp = a; // Eltároljuk 'a' értékét
    a = b; // 'a' megkapja 'b' értékét
    b = temp; // 'b' megkapja 'a' értékét
}
```

```

    b = temp;          // 'b' megkapja az eredeti 'a' értékét
}

int első = 10;
int második = 20;

Console.WriteLine($"Előtte: első={első}, második={második}");
Csere(ref első, ref második); // Kicseréljük őket
Console.WriteLine($"Utána: első={első}, második={második}");
// Előtte: első=10, második=20
// Utána: első=20, második=10

// Több érték "visszaadása"
static void KorbenNoveles(ref int szám1, ref int szám2, ref int szám3)
{
    szám1 += 1;
    szám2 += 1;
    szám3 += 1;
}

int a = 5, b = 10, c = 15;
KorbenNoveles(ref a, ref b, ref c);
Console.WriteLine($"{{a}}, {{b}}, {{c}}"); // 6, 11, 16

```

out paraméter - Kimeneti paraméter

Az **out** paraméter hasonló a **ref**-hez, de **nem kell előre inicializálni**, és a módszernek **kötelező értéket adnia** neki.

```

// out PARAMÉTER - a módszer FELTÖLTÍ az értékét
static void Osszegzes(int a, int b, out int osszeg, out int kulonbseg)
{
    // A módszernek KÖTELEZŐ értéket adni az out paramétereknek!
    osszeg = a + b;
    kulonbseg = a - b;
    // Ha nem adnánk értéket, fordítási HIBA lenne!
}

// HASZNÁLAT:
int ossz, kul; // NEM kell inicializálni!

Osszegzes(10, 5, out ossz, out kul);
// A módszer feltölti az "ossz" és "kul" értékét

Console.WriteLine($"Összeg: {{ossz}}"); // 15
Console.WriteLine($"Különbség: {{kul}}"); // 5

// C# 7.0+ : inline deklaráció
Osszegzes(20, 8, out int o, out int k);
Console.WriteLine($"Összeg: {{o}}, Különbség: {{k}}"); // 28, 12

```

```
// Gyakorlati példa: Szám értelmezés (parsing)
static bool ProbalMegParsolni(string szoveg, out int szam)
{
    // Megpróbáljuk a szöveget számmá alakítani
    try
    {
        szam = int.Parse(szoveg);
        return true; // Sikeres!
    }
    catch
    {
        szam = 0; // Sikertelen - 0-t adunk vissza
        return false; // Jelezzük a hibát
    }
}

// Használat
if (ProbalMegParsolni("123", out int eredmeny))
{
    Console.WriteLine($"Sikeres! Szám: {eredmeny}");
}
else
{
    Console.WriteLine("Nem sikerült számmá alakítani!");
}

// Beépített TryParse metódusok hasonlóan működnek
if (int.TryParse("456", out int szam))
{
    Console.WriteLine($"Szám: {szam}");
}

// Több érték visszaadása egyszerűen
static void KoordinataSzamitas(double x, double y,
                                out double tavolsag,
                                out double szog)
{
    // Távolság origótól
    tavolsag = Math.Sqrt(x * x + y * y);

    // Szög (radiánban)
    szog = Math.Atan2(y, x);
}

KoordinataSzamitas(3, 4, out double t, out double sz);
Console.WriteLine($"Távolság: {t:F2}, Szög: {sz:F2}");

// ref vs out KÜLÖNBESÉG:
// ref: - Kell inicializálni hívás előtt
//       - A metódus OLVASHATJA az értékét
//       - A metódus MÓDOSÍTHATJA az értékét

// out: - NEM kell inicializálni
```

```
//      - A metókus NEM OLVASHATJA a régi értékét  
//      - A metódusnak KÖTELEZŐ új értéket adni
```

params paraméter - Változó számú paraméter

A **params** kulcsszóval **tetszőleges számú** paramétert fogadhatunk.

```
// params - változó számú paraméter  
static int OsszeadMindet(params int[] szamok)  
{  
    // A "szamok" egy tömb, ami tartalmazza az összes átadott értéket  
    int osszeg = 0;  
  
    foreach (int szam in szamok)  
    {  
        osszeg += szam;  
    }  
  
    return osszeg;  
}  
  
// HASZNÁLAT - tetszőleges számú paramétert adhatunk át!  
int a = OsszeadMindet(5);           // 5  
int b = OsszeadMindet(10, 20);      // 30  
int c = OsszeadMindet(1, 2, 3, 4, 5); // 15  
int d = OsszeadMindet(10, 20, 30, 40, 50, 60); // 210  
  
Console.WriteLine($"a={a}, b={b}, c={c}, d={d}");  
  
// Akár üres is lehet!  
int ures = OsszeadMindet(); // 0  
  
// Vagy tömbként is átadhatjuk  
int[] tomb = { 5, 10, 15, 20 };  
int e = OsszeadMindet(tomb); // 50  
  
// Gyakorlati példa: Átlag számítás  
static double AtlagSzamitas(params double[] szamok)  
{  
    if (szamok.Length == 0)  
        return 0;  
  
    double osszeg = 0;  
    foreach (double szam in szamok)  
    {  
        osszeg += szam;  
    }  
  
    return osszeg / szamok.Length;  
}
```

```
double atlag1 = AtlagSzamitas(5, 10, 15);           // 10
double atlag2 = AtlagSzamitas(80, 90, 85, 95, 78); // 85.6

// Szövegek összefűzése
static string Osszefuz(string elvalaszto, params string[] szovegek)
{
    // Normál paraméter + params paraméter
    return string.Join(elvalaszto, szovegek);
}

string eredmeny1 = Osszefuz(", ", "alma", "körte", "barack");
Console.WriteLine(eredmeny1); // alma, körte, barack

string eredmeny2 = Osszefuz(" | ", "egy", "kettő", "három", "négy");
Console.WriteLine(eredmeny2); // egy | kettő | három | négy

// FONTOS SZABÁLYOK:
// 1. Csak EGY params paraméter lehet
// static void Rossz(params int[] a, params int[] b) { } // HIBA!

// 2. A params paraméternek UTOLSÓNAK kell lennie
// static void Rossz2(params int[] a, int b) { } // HIBA!
// static void Jo(int b, params int[] a) { } // OK!

// Maximum keresése
static int MaxKereses(params int[] szamok)
{
    if (szamok.Length == 0)
        throw new ArgumentException("Legalább egy szám kell!");

    int max = szamok[0];

    foreach (int szam in szamok)
    {
        if (szam > max)
            max = szam;
    }

    return max;
}

int legnagyobb = MaxKereses(45, 23, 78, 12, 90, 34);
Console.WriteLine($"Legnagyobb: {legnagyobb}"); // 90
```

Method Overloading - Metódus túlterhelés

Ugyanaz a metódusnév, de **különböző paraméterek**. A C# automatikusan kiválasztja a megfelelő verziót.

```
// UGYANAZ A NÉV, de különböző paraméterek

// 1. verzió - két int
static int Osszead(int a, int b)
{
    Console.WriteLine("int verzió hívva");
    return a + b;
}

// 2. verzió - három int
static int Osszead(int a, int b, int c)
{
    Console.WriteLine("3 int verzió hívva");
    return a + b + c;
}

// 3. verzió - két double
static double Osszead(double a, double b)
{
    Console.WriteLine("double verzió hívva");
    return a + b;
}

// 4. verzió - két string (összefűzés)
static string Osszead(string a, string b)
{
    Console.WriteLine("string verzió hívva");
    return a + b;
}

// HASZNÁLAT - a C# automatikusan kiválasztja a megfelelő verziót
int eredmeny1 = Osszead(5, 10);           // int verzió → 15
int eredmeny2 = Osszead(5, 10, 15);       // 3 int verzió → 30
double eredmeny3 = Osszead(5.5, 10.2);    // double verzió → 15.7
string eredmeny4 = Osszead("Hello", "World"); // string verzió → "HelloWorld"

// A PARAMÉTEREK SZÁMA vagy TÍPUSA különböző kell legyen!

// HELYES példák:
static void Kiir(int szam) { }
static void Kiir(string szoveg) { }        // OK - típus más
static void Kiir(int a, int b) { }        // OK - paraméterek száma más
static void Kiir(double szam) { }         // OK - típus más

// HELYTELEN példák:
// static void Kiir(int x) { }              // HIBA! Már van Kiir(int)
// static int Kiir(int szam) { }           // HIBA! A visszatérési típus nem
// számít!

// Gyakorlati példa: Területszámítás
static double Terulet(double sugar)
{
    // Kör területe
```

```
        return Math.PI * sugar * sugar;
    }

    static double Terulet(double a, double b)
    {
        // Téglalap területe
        return a * b;
    }

    static double Terulet(double a, double b, double c)
    {
        // Háromszög területe (Heron-képlet)
        double s = (a + b + c) / 2;
        return Math.Sqrt(s * (s - a) * (s - b) * (s - c));
    }

    // Használat
    double kor = Terulet(5);           // Kör: sugár = 5
    double teglalap = Terulet(4, 6);    // Téglalap: 4x6
    double haromszog = Terulet(3, 4, 5); // Háromszög: 3,4,5 oldalú

    Console.WriteLine($"Kör: {kor:F2}");           // 78.54
    Console.WriteLine($"Téglalap: {teglalap}");    // 24
    Console.WriteLine($"Háromszög: {haromszog}");  // 6

    // WriteLine is túlterhelt!
    Console.WriteLine(42);           // int verzió
    Console.WriteLine("szöveg");     // string verzió
    Console.WriteLine(3.14);         // double verzió
    Console.WriteLine(true);         // bool verzió
```

Rekurzió - Metódus hívja önmagát

Egy metódus **meghívhatja saját magát**. Ez a rekurzió.

```
// EGYSZERŰ REKURZÍV PÉLDA: Visszaszámlálás

static void Visszaszamlalas(int n)
{
    // 1. ALAPESET (KILÉPÉSI FELTÉTEL) - kötelező!
    if (n == 0)
    {
        Console.WriteLine("Start!");
        return; // Kilépünk, nem hívjuk tovább
    }

    // 2. REKURZÍV ESET
    Console.WriteLine(n);
    Visszaszamlalas(n - 1); // ÖNMAGA HÍVÁSA kisebb értékkel!
}
```

```
// Használat
Visszaszamlalas(5);
// Kimenet:
// 5
// 4
// 3
// 2
// 1
// Start!

// MI TÖRTÉNIK A HÁTTÉRBE?
// Visszaszamlalas(5)
//   → kiírja "5"
//   → meghívja Visszaszamlalas(4)
//     → kiírja "4"
//     → meghívja Visszaszamlalas(3)
//       → kiírja "3"
//       → meghívja Visszaszamlalas(2)
//         → kiírja "2"
//         → meghívja Visszaszamlalas(1)
//           → kiírja "1"
//           → meghívja Visszaszamlalas(0)
//             → kiírja "Start!"
//             → return (vége)

// FAKTORIÁLIS SZÁMÍTÁS rekurzívan
//  $n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$ 
// Példa:  $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ 

static int Faktorial(int n)
{
    // ALAPESET:  $0! = 1$ ,  $1! = 1$ 
    if (n <= 1)
    {
        return 1;
    }

    // REKURZÍV ESET:  $n! = n \times (n-1)!$ 
    return n * Faktorial(n - 1);
}

int eredmeny = Faktorial(5);
Console.WriteLine($"5! = {eredmeny}"); // 120

// FIBONACCI SOROZAT rekurzívan
// 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
// Minden szám az előző kettő összege

static int Fibonacci(int n)
{
    // ALAPESETEK
    if (n == 0) return 0;
    if (n == 1) return 1;
```

```
// REKURZÍV ESET:  $F(n) = F(n-1) + F(n-2)$ 
return Fibonacci(n - 1) + Fibonacci(n - 2);
}

// Első 10 Fibonacci szám
for (int i = 0; i < 10; i++)
{
    Console.WriteLine($"{Fibonacci(i)} ");
}
// Kimenet: 0 1 1 2 3 5 8 13 21 34

// REKURZIÓ vs CIKLUS
// Ugyanaz a feladat ciklussal:
static int FaktorialCiklus(int n)
{
    int eredmeny = 1;
    for (int i = 2; i <= n; i++)
    {
        eredmeny *= i;
    }
    return eredmeny;
}

// Melyik a jobb?
// Rekurzió: - Elegánsabb, olvashatóbb
//           - Lassabb (metódushívások költsége)
//           - Stack overflow veszélye nagy értékeknél
// Ciklus:   - Gyorsabb
//           - Memóriahatékonyabb
//           - Néha nehezebben érthető

// VIGYÁZZ! Végtelen rekurzió ha nincs ALAPESET:
// static void Vegtelen(int n)
// {
//     Console.WriteLine(n);
//     Vegtelen(n - 1); // NINCS kilépési feltétel → STACK OVERFLOW!
// }
```

Lokális és globális változók

A változók **láthatósága** (scope) attól függ, hol deklaráltuk őket.

```
// GLOBÁLIS VÁLTOZÓ (osztályszintű)
class Program
{
    static int globalValtozo = 100; // Minden metódus látja

    static void Main(string[] args)
    {
```

```
// LOKÁLIS VÁLTOZÓ (metódus szintű)
int lokalValtozo = 10; // Csak a Main-ben látható

Console.WriteLine($"Globális: {globalValtozo}"); // OK
Console.WriteLine($"Lokális: {lokalValtozo}"); // OK

MasikMetodus();

// int x = lokalValtozo; // OK, még a Main-ben vagyunk
}

static void MasikMetodus()
{
    Console.WriteLine($"Globális itt is: {globalValtozo}"); // OK

    // Console.WriteLine(lokalValtozo); // HIBA! Nem látható itt!
    // A lokalValtozo csak a Main-ben létezik!
}
}

// RÉSZLETESEBB PÉLDA:

class Program2
{
    // Globális változó (osztályszintű)
    static string programNev = "Szuper Program";
    static int hivas = 0;

    static void Main(string[] args)
    {
        // Lokális változó a Main-ben
        int szam = 5;

        Console.WriteLine($"Program: {programNev}");

        Metodus1();
        Metodus2(szam); // Átadjuk paraméterként

        Console.WriteLine($"Metódushívások: {hivas}");
    }

    static void Metodus1()
    {
        // Saját lokális változó
        int saját = 20;

        // Látja a globális változót
        Console.WriteLine($"Globális: {programNev}");

        // Módosíthatja a globális változót
        hivas++;

        // Console.WriteLine(szam); // HIBA! A szam csak a Main-ben létezik
        // Console.WriteLine(saját2); // HIBA! A saját2 még nem létezik
    }
}
```

```
}

static void Metodus2(int parameter)
{
    // Saját lokális változó (más név mint Metodus1-ben!)
    int saját2 = 30;

    // A parameter is lokális változó (csak itt látható)
    Console.WriteLine($"Kapott paraméter: {parameter}");

    hivas++;

    // Console.WriteLine(saját); // HIBA! A saját csak Metodus1-ben létezik
}
}

// BLOKK SCOPE (kapcsos zárójeleken belül)
static void BlokkPelda()
{
    int x = 10; // Látható az egész metódusban

    if (x > 5)
    {
        int y = 20; // Csak az if blokkon belül látható!
        Console.WriteLine($"x={x}, y={y}"); // OK
    }

    Console.WriteLine($"x={x}"); // OK
    // Console.WriteLine($"y={y}"); // HIBA! Az y már nem látható!

    for (int i = 0; i < 3; i++)
    {
        int z = 30; // Csak a for blokkon belül
    }

    // Console.WriteLine(i); // HIBA! Az i már nem létezik
    // Console.WriteLine(z); // HIBA! A z már nem létezik
}

// SHADOWING - Ugyanaz a név, de más scope
static int globalSzam = 100;

static void ShadowPelda()
{
    int globalSzam = 50; // LOKÁLIS változó, elfedi a globális!
    Console.WriteLine(globalSzam); // 50 (a lokális)

    // A globális elérhető így:
    Console.WriteLine(Program.globalSzam); // 100
}
```

Expression-bodied members (C# 6.0+)

Rövid metódusokat **kifejezés formában** írhatunk.

```
// HAGYOMÁNYOS METÓDUS
static int KetszereseRegi(int x)
{
    return x * 2;
}

// EXPRESSION-BODIED (kifejezés test)
static int Ketszerese(int x) => x * 2;
// A "=> " azt jelenti: "return"

// Mindkettő ugyanazt csinálja, de a második rövidebb!

// Használat
int eredmeny = Ketszerese(5); // 10

// TÖBB PÉLDA:

// Normál verzió
static bool ParosENormalis(int szam)
{
    return szam % 2 == 0;
}

// Expression-bodied
static bool ParosE(int szam) => szam % 2 == 0;

// Normál verzió
static string TeljesNevNormalis(string vezetek, string kereszt)
{
    return $"{vezetek} {kereszt}";
}

// Expression-bodied
static string TeljesNev(string vezetek, string kereszt)
    => $"{vezetek} {kereszt}";

// void metódusnál is használható (C# 7.0+)
static void Udvozles(string nev) => Console.WriteLine($"Hello, {nev}!");

// Többsoros kifejezésnél érdemes többsorban írni
static double TeruletSzamitas(double sugar) =>
    Math.PI * sugar * sugar;

// KOMPLEX PÉLDA
static int AbszolutErtek(int szam) => szam < 0 ? -szam : szam;

// MIKOR HASZNÁLJUK?
// - Egyszerű, egysoros metódusoknál
```

```
// - Ha csak egy kifejezés van
// - Olvashatóbb lesz a kód

// MIKOR NE HASZNÁLJUK?
// - Ha több sor logika van
// - Ha nehezebben olvasható lesz
// - Ha több utasítás van

// ROSSZ PÉLDA (túl bonyolult expression-bodied-nek):
// static int Valami(int x) =>
//     x > 10 ? x * 2 : x < 5 ? x + 10 : x * x; // Nehezen olvasható!

// JOBB lenne normál formában:
static int Valami(int x)
{
    if (x > 10)
        return x * 2;
    else if (x < 5)
        return x + 10;
    else
        return x * x;
}
```

Gyakorlati nagy példák

1. Számológép program

```
using System;

class Szamologep
{
    static void Main(string[] args)
    {
        // Számológép fő program
        Console.WriteLine("=== EGYSZERŰ SZÁMOLÓGÉP ===\n");

        // Számok bekérése
        Console.Write("Add meg az első számot: ");
        double szam1 = double.Parse(Console.ReadLine());

        Console.Write("Add meg a második számot: ");
        double szam2 = double.Parse(Console.ReadLine());

        // Művelet bekérése
        Console.Write("Művelet (+, -, *, /): ");
        string muvelet = Console.ReadLine();

        // Számítás végrehajtása (metódus hívás)
        double eredmeny = Szamitas(szam1, szam2, muvelet);
    }
}
```

```
// Eredmény kiírása
if (!double.IsNaN(eredmeny)) // NaN = Not a Number (hiba)
{
    Console.WriteLine($"Eredmény: {szam1} {muvelet} {szam2} = {eredmeny}");
}

// Számítást végző metódus
static double Szamitas(double a, double b, string muvelet)
{
    // switch kifejezéssel (C# 8.0+)
    return muvelet switch
    {
        "+" => Osszead(a, b),
        "-" => Kivon(a, b),
        "*" => Szoroz(a, b),
        "/" => Oszt(a, b),
        _ => HibaUzenet() // _ = default eset
    };
}

// Összeadás
static double Osszead(double a, double b)
{
    return a + b;
}

// Kivonás
static double Kivon(double a, double b)
{
    return a - b;
}

// Szorzás
static double Szoroz(double a, double b)
{
    return a * b;
}

// Osztás (hibakezeléssel)
static double Oszt(double a, double b)
{
    // Ellenőrizzük, hogy nem-e 0-val osztunk
    if (b == 0)
    {
        Console.WriteLine("HIBA: Nullával nem lehet osztani!");
        return double.NaN; // Jelezzük a hibát
    }
    return a / b;
}

// Hibaüzenet
static double HibaUzenet()
```

```
{  
    Console.WriteLine("HIBA: Ismeretlen művelet!");  
    return double.NaN;  
}  
}
```

2. Felhasználó regisztráció validációval

```
using System;  
  
class Regisztracio  
{  
    static void Main(string[] args)  
    {  
        Console.WriteLine("=== REGISZTRÁCIÓ ===\n");  
  
        // Felhasználónév bekérése  
        string felhasznalo = FelhasznaloBekeresdes();  
  
        // Jelszó bekérése  
        string jelszo = JelszoBekeresdes();  
  
        // Email bekérése  
        string email = EmailBekeresdes();  
  
        // Sikeres regisztráció  
        SikeresRegisztracio(felhasznalo, email);  
    }  
  
    // Felhasználónév bekérése validációval  
    static string FelhasznaloBekeresdes()  
    {  
        while (true) // Addig kérjük, amíg jó nem lesz  
        {  
            Console.Write("Felhasználónév (min. 3 karakter): ");  
            string nev = Console.ReadLine();  
  
            // Validálás  
            if (ErvenyesFelhasznalo(nev))  
            {  
                return nev; // Kilépünk, visszaadjuk  
            }  
  
            // Ha nem jó, folytatjuk a ciklust  
            Console.WriteLine("Hibás felhasználónév! Próbáld újra.\n");  
        }  
    }  
  
    // Felhasználónév validálása  
    static bool ErvenyesFelhasznalo(string nev)  
    {  

```

```
// Ellenőrzések
if (string.IsNullOrEmpty(nev))
{
    Console.WriteLine("- A név nem lehet üres!");
    return false;
}

if (nev.Length < 3)
{
    Console.WriteLine("- Legalább 3 karakter kell!");
    return false;
}

if (nev.Contains(" "))
{
    Console.WriteLine("- Nem tartalmazhat szóközt!");
    return false;
}

return true; // Minden rendben
}

// Jelszó bekérése validációval
static string JelszoBekeres()
{
    while (true)
    {
        Console.Write("Jelszó (min. 6 karakter): ");
        string jelszo1 = Console.ReadLine();

        // Jelszó erősségének ellenőrzése
        if (!ErvenyesJelszo(jelszo1))
        {
            Console.WriteLine("Hibás jelszó! Próbáld újra.\n");
            continue;
        }

        // Jelszó megerősítése
        Console.Write("Jelszó újra: ");
        string jelszo2 = Console.ReadLine();

        if (jelszo1 == jelszo2)
        {
            return jelszo1;
        }

        Console.WriteLine("A két jelszó nem egyezik!\n");
    }
}

// Jelszó validálása
static bool ErvenyesJelszo(string jelszo)
{
    if (string.IsNullOrEmpty(jelszo))
```

```
{
    Console.WriteLine("- A jelszó nem lehet üres!");
    return false;
}

if (jelszo.Length < 6)
{
    Console.WriteLine("- Legalább 6 karakter kell!");
    return false;
}

// Számjegy ellenőrzés
bool vanSzamjegy = false;
foreach (char c in jelszo)
{
    if (char.IsDigit(c))
    {
        vanSzamjegy = true;
        break;
    }
}

if (!vanSzamjegy)
{
    Console.WriteLine("- Tartalmaznia kell számjegyet!");
    return false;
}

return true;
}

// Email bekérése validációval
static string EmailBekeres()
{
    while (true)
    {
        Console.Write("Email cím: ");
        string email = Console.ReadLine();

        if (ErvenyesEmail(email))
        {
            return email;
        }

        Console.WriteLine("Hibás email! Próbáld újra.\n");
    }
}

// Email validálása (egyszerűsített)
static bool ErvenyesEmail(string email)
{
    if (string.IsNullOrEmpty(email))
    {
        Console.WriteLine("- Az email nem lehet üres!");
    }
}
```

```

        return false;
    }

    if (!email.Contains("@"))
    {
        Console.WriteLine("- Hiányzik a @ jel!");
        return false;
    }

    if (!email.Contains("."))
    {
        Console.WriteLine("- Hiányzik a pont!");
        return false;
    }

    // @ jel után legyen még valami
    int kukacIndex = email.IndexOf('@');
    if (kukacIndex == email.Length - 1)
    {
        Console.WriteLine("- A @ jel után kell valami!");
        return false;
    }

    return true;
}

// Sikeres regisztráció üzenet
static void SikeresRegisztracio(string felhasznalo, string email)
{
    Console.WriteLine("\n" + new string('=', 40));
    Console.WriteLine("✓ SIKERES REGISZTRÁCIÓ!");
    Console.WriteLine(new string('=', 40));
    Console.WriteLine($"Felhasználó: {felhasznalo}");
    Console.WriteLine($"Email: {email}");
    Console.WriteLine($"Regisztráció ideje: {DateTime.Now}");
    Console.WriteLine(new string('=', 40));
}
}

```

3. Játék - Szám kitalálás

```

using System;

class SzamKitalalo
{
    // Globális változók
    static Random rnd = new Random();
    static int probalkozasok = 0;
    static int legjobb = int.MaxValue; // Legjobb eredmény

    static void Main(string[] args)
    {
    }
}

```

```
{
    bool ujraJatszik = true;

    while (ujraJatszik)
    {
        // Játék indítása
        JatekIndit();

        // Újra játszik?
        ujraJatszik = KerdezUjJatek();
    }

    // Kilépés
    Console.WriteLine("\nKöszönjük a játékot!");
    StatisztikaKiir();
}

// Játék indítása
static void JatekIndit()
{
    Console.Clear();
    Fejlec();

    // Véletlenszerű szám generálása (1-100)
    int titkosSzam = rnd.Next(1, 101);
    probalkozasok = 0;

    Console.WriteLine("Gondoltam egy számra 1 és 100 között.");
    Console.WriteLine("Találd ki!\n");

    // Játék menete
    bool talalt = false;
    while (!talalt)
    {
        probalkozasok++;

        // Tipp bekérése
        int tipp = TippBeker();

        // Ellenőrzés
        talalt = EllenorizTipp(tipp, titkosSzam);
    }

    // Győzelem
    GyozelemUzenet();
}

// Fejléc kiírása
static void Fejlec()
{
    Console.WriteLine("
");
    Console.WriteLine("
    SZÁM KITALÁLÓS JÁTÉK
");
    Console.WriteLine("
");
    Console.WriteLine();
}
```

```
}

// Tipp bekérése validációval
static int TippBeker()
{
    while (true)
    {
        Console.WriteLine($"{probalkozasok}. próbálkozás - Tipped: ");
        string input = Console.ReadLine();

        // Validálás
        if (int.TryParse(input, out int tipp))
        {
            if (tipp >= 1 && tipp <= 100)
            {
                return tipp;
            }
            Console.WriteLine("1 és 100 közötti számot adj meg!\n");
        }
        else
        {
            Console.WriteLine("Érvényes számot adj meg!\n");
        }
    }
}

// Tipp ellenőrzése
static bool EllenorizTipp(int tipp, int helyes)
{
    if (tipp == helyes)
    {
        Console.WriteLine("✓ ELTALÁLTAD!");
        return true;
    }
    else if (tipp < helyes)
    {
        // Segítség - mennyivel tér el
        int kulonbseg = helyes - tipp;
        string uzenet = KulonbsegUzenet(kulonbseg);
        Console.WriteLine($"↑ Nagyobb! {uzenet}");
    }
    else
    {
        int kulonbseg = tipp - helyes;
        string uzenet = KulonbsegUzenet(kulonbseg);
        Console.WriteLine($"↓ Kisebb! {uzenet}");
    }

    Console.WriteLine();
    return false;
}

// Különbség alapján segítő üzenet
static string KulonbsegUzenet(int kulonbseg)
```

```

{
    if (kulonbseg <= 5)
        return "(Nagyon közel!)";
    else if (kulonbseg <= 10)
        return "(Közel vagy!)";
    else if (kulonbseg <= 20)
        return "(Meleg)";
    else
        return "(Hideg)";
}

// Győzelem üzenet
static void GyozelemUzenet()
{
    Console.WriteLine();
    Console.WriteLine(new string('=', 40));
    Console.WriteLine($"Gratulálunk! {probalkozasok} próbálkozásból találtad
ki!");

    // Legjobb eredmény frissítése
    if (probalkozasok < legjobb)
    {
        legjobb = probalkozasok;
        Console.WriteLine("Ez az új rekord!");
    }

    // Értékelés
    string ertekeles = ErtekelesKiszamit(probalkozasok);
    Console.WriteLine($"Értékelés: {ertekeles}");
    Console.WriteLine(new string('=', 40));
    Console.WriteLine();
}

// Értékelés a próbálkozások alapján
static string ErtekelesKiszamit(int probak)
{
    if (probak <= 5)
        return "ZSENIÁLIS! 🏆";
    else if (probak <= 8)
        return "Nagyon jó! ★";
    else if (probak <= 12)
        return "Jó! 🥇";
    else if (probak <= 15)
        return "Közepes 😊";
    else
        return "Gyakorolj még! 🤔";
}

// Új játék kérdése
static bool KerdezUjJatek()
{
    Console.Write("Játszol még egyet? (i/n): ");
    string valasz = Console.ReadLine().ToLower();
    return valasz == "i" || valasz == "igen";
}

```

```
    }

    // Statisztika kiírása
    static void StatisztikaKiir()
    {
        if (legjobb != int.MaxValue)
        {
            Console.WriteLine($"\\nLegjobb eredményed: {legjobb} próbálkozás");
        }
    }
}
```

Összefoglalás

Metódus szerkezet

```
[static] [visszatérési típus] [Metódusnév]([paraméterek])
{
    // metódus törzse
    [return érték;]
}
```

Visszatérési típusok

- **void**: Nem ad vissza semmit
- **int, double, string, bool, stb.**: Adott típusú értéket ad vissza
- **return**: Visszaadja az értéket és kilép

Paraméterek

- **Normál**: Érték másolat (nem változik az eredeti)
- **ref**: Referencia átadás (változik az eredeti, előtte inicializálni kell)
- **out**: Kimeneti paraméter (változik az eredeti, nem kell inicializálni)
- **params**: Változó számú paraméter (tömb formájában)

Speciális

- **Alapértelmezett paraméterek**: Opcionális értékek
- **Nevesített paraméterek**: Név szerinti átadás
- **Method overloading**: Ugyanaz a név, más paraméterek
- **Rekurzió**: Metódus hívja önmagát
- **Expression-bodied**: Rövidített szintaxis

Fontos szabályok

1. **Egy feladat, egy metódus** - ne csináljon túl sokat
2. **Érthető nevek** - azonnal látható, mit csinál

3. **Paraméterek száma** - max 3-4, több esetén használj objektumot
 4. **Dokumentáció** - kommentezd a bonyolultabb részeket
 5. **return mindig kilép** - alatta levő kód nem fut le
-

C# Alapok Tananyag - 8. Fejezet